Instalarea Turbo Assembler

In vederea rulării TASM pentru orice sistem diferit de DOS, avem nevoie în principiu, de un emulator de DOS. Pentru Windows poate fi folosit DOSBox, un utilitar ce poate fi download-at de pe siteul www.dosbox.com. Este de preferat ca DOSBox sa fie instalat in locatia implicită (`C:\Program Files (x86)\DOSBox-...)`.

Următorul pas constă în download-area arhivei tasm.zip și dezarhivarea acesteia într-un director pregatit pentru lucru (e.g. `E:\PrjMele\ASM`).

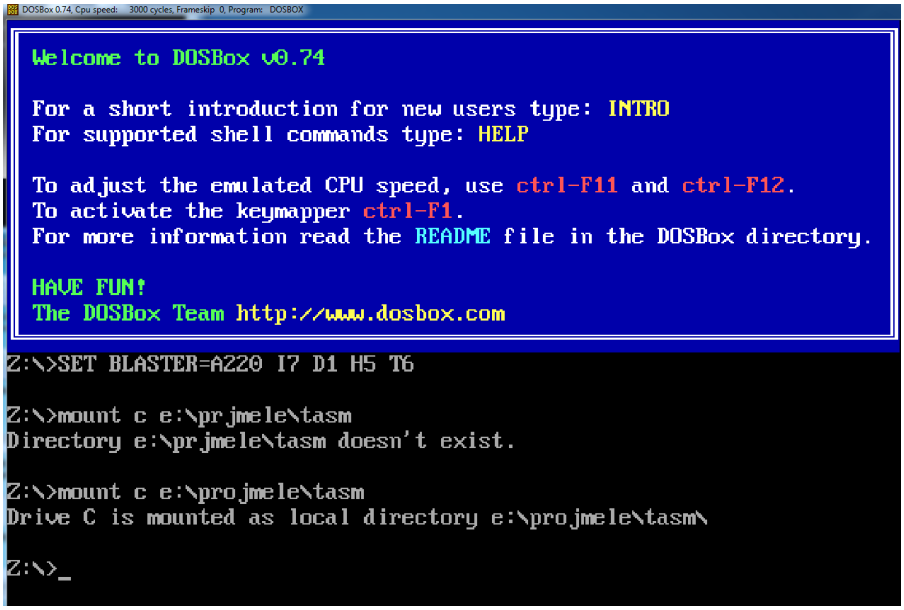
Folosind shortcut-ul apărut după instalarea DOSBox-ului,

se rulează emulatorul DOS și se montează locația TASM-ului, comană prin care driverul c din emulator devine path-ul de lucru cu TASM.

```
Z:\> mount c E:\PrjMele\ASM\
```

În urma acestui proces veți primi un mesaj de confirmare:

”`Drive C is mounted as local directory E:\PrjMele\ASM\`„



Din acest moment ar trebui să aveți posibilitatea asamblării și link-ării programelor (de exemplu programul exercitiu1.asm)

OBSERVAŢIE

Acum avem creat un fisier de configurare al DosBox într-un director de genul C:\Users\...AppData\Local\DOSBox\dosbox-0.74.conf care poate seta proprietăţile fereastrei:

```
fullscreen=false
fulldouble=false
fullresolution=1440x900
windowresolution=1440x900
output=ddraw
autolock=true
sensitivity=100
```

Compilarea și rularea unei aplicații ASM

Turbo Assembler-ul are un help inline care se poate apela cu parametrul /h:

```
> tasm /h
```

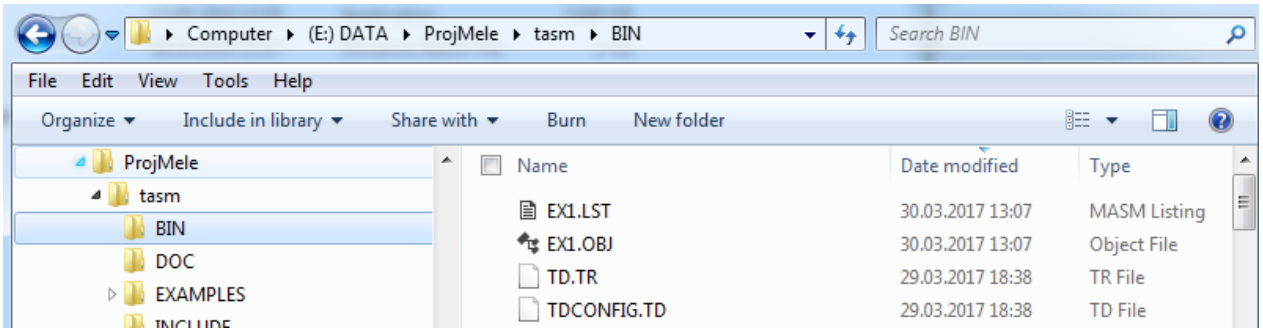
Asamblarea se poate executa cu comanda

```
> tasm <nume_sursa> /zi /la
```

Unde /zi: produce toate informaţiile de debug şi /la: generează listing-ul expandat.

```
C:\BIN>tasm ex1.asm /zi /la
Turbo Assembler  Version 4.1  Copyright (c) 1988, 1996 Borland International

Assembling file:   ex1.asm
Error messages:    None
Warning messages:  None
Passes:            1
Remaining memory:  460k


C:\BIN>_
```

Rezultă generarea a două fişiere OBJ şi LST:



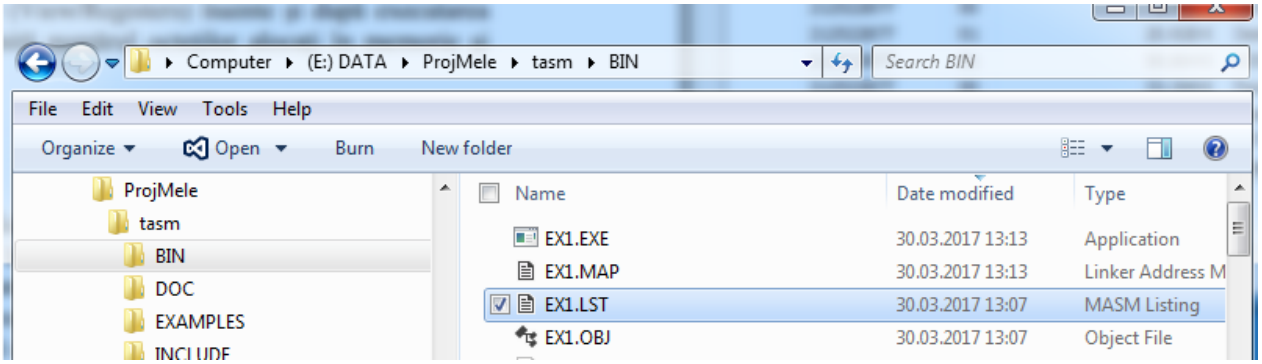
Linkeditarea se face cu comanda:

```
> tlink /v <nume_sursa>
```

Unde /v: va produce toate informaţiile symbol pentru debug

```
C:\BIN>tlink /v ex1.obj
Turbo Link  Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International

C:\BIN>_
```

Şi va produce cele două fişiere EXE şi MAP.



În acest stadiu va trebui sa inspectaţi cele două fişiere tip ASCII produse până acum.

Execuţia se face prin simpla apelare a executabilului EX1Ş

```
> ex1<.exe>
```

Depanarea unei aplicații ASM

Pentru a intra in sesiunea de debugging pe aplicaţia EX1, se apelează TD (Turbo Debugging):

```
> td ex1.exe
```

```
C:\BIN>td ex1.exe
Turbo Debugger Version 5.0 Copyright (c) 1988,96 Borland International
```

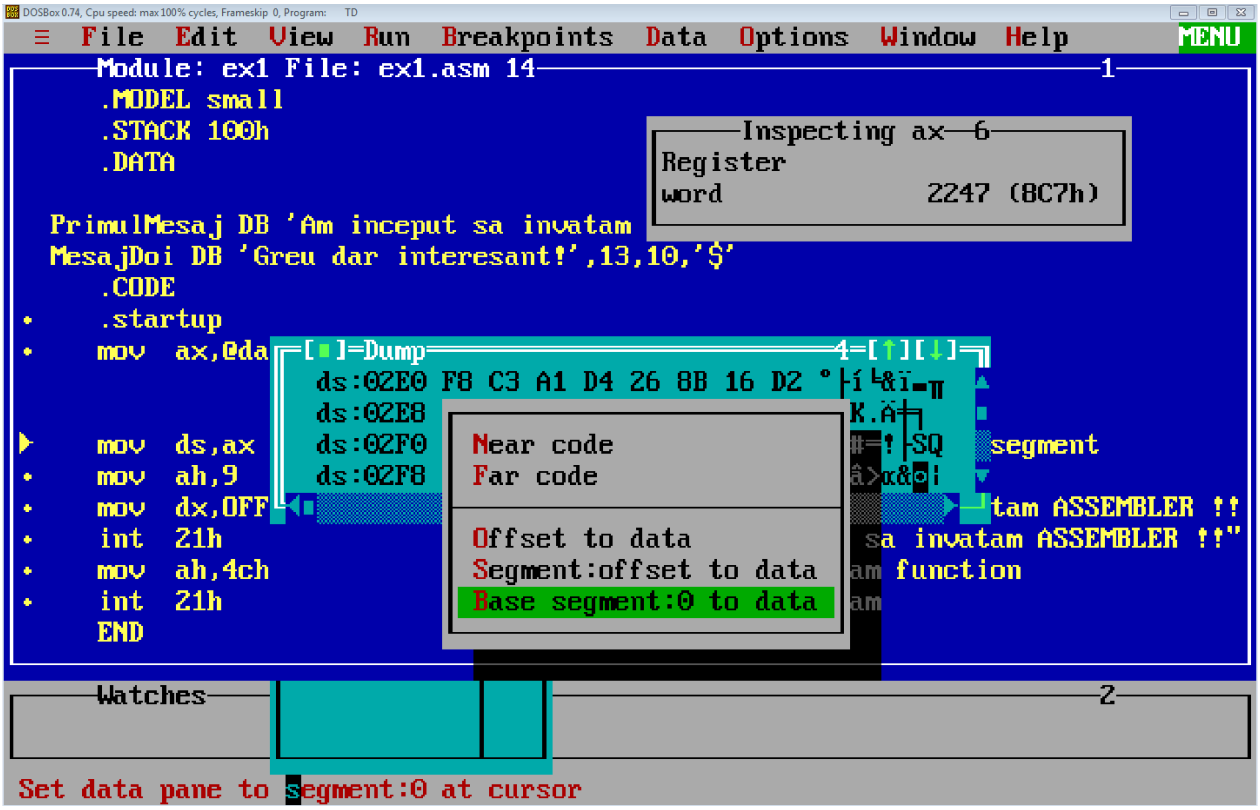
Acum se va deschide aplicaţia Turbo Debugger:

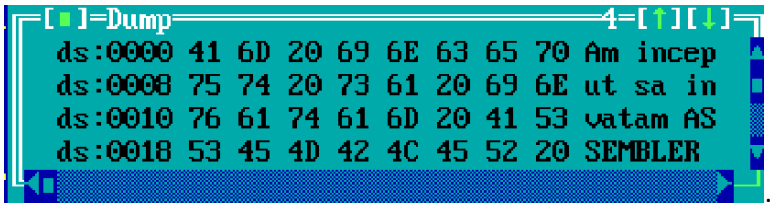
```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program:    TD
 ≡  File  Edit  View  Run  Breakpoints  Data  Options  Window  Help          READY
[■]=Module: ex1 File: ex1.asm 14                                          1=[↑][↓]
  ; Turbo Assembler    Copyright (c) 1988, 1991 By Borland International, Inc.

  ; Exercitiu1.ASM - Afiseaza mesajul "Invatam ASSEMBLER !!"

  ;

     .MODEL small
     .STACK 100h
     .DATA

  PrimulMesaj DB 'Am inceput sa invatam ASSEMBLER !!',13,10,'$'
  MesajDoi DB 'Greu dar interesant!',13,10,'$'
     .CODE
▶ _  .startup
•    mov  ax,@data            ; cu debug mergeti la adresa data
                            ; si cu offset-ul respectiv
                            ; inspectati sirul
•    mov  ds,ax                   ;set DS to point to the data segment
     Watches                                                              2
MesajDoi                    byte [20] "Greu dar interesant!"
F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu
```

Din menu alegem Data -> Inspect şi tastăm ax, va rezulta:

```
DOSBox 0.74, Cpu speed: max 100% cycles, Frameskip 0, Program:    TD
 ≡  File  Edit  View  Run  Breakpoints  Data  Options  Window  Help          READY
     Module: ex1 File: ex1.asm 14                                             1
     .MODEL small
     .STACK 100h                          [■]=Inspecting ax=6=[↑][↓]
     .DATA                                Register
                                          word              2247 (8C7h)
  PrimulMesaj DB 'Am inceput sa invatam
  MesajDoi DB 'Greu dar interesant!',13,10,'$'
     .CODE
•    .startup
•    mov  ax,@data            ; cu debug mergeti la adresa data
                            ; si cu offset-ul respectiv
                            ; inspectati sirul
▶    mov  ds,ax                   ;set DS to point to the data segment
•    mov  ah,9                    ;DOS print string function
•    mov  dx,OFFSET PrimulMesaj   ;point to "Am inceput sa invatam ASSEMBLER !!
•    int  21h                     ;display "Am inceput sa invatam ASSEMBLER !!"
•    mov  ah,4ch                  ;DOS terminate program function
•    int  21h                     ;terminate the program
     END
     Watches                                                              2
F1-Help F2-Bkpt F3-Mod F4-Here F5-Zoom F6-Next F7-Trace F8-Step F9-Run F10-Menu
```

Apoi facem „dump” pe locatia de la adresa 0x8C7 pentru a vedea datele:



Introducem segmentul şi offsetul pentru locaţia de memorie care pointează la variabila noastră:

Şi vom putea inspecta şirul ce reprezintă variabila căutată: